Makoya — Project Brief

Makoya

June 30, 2026

CONTENTS

Makoya — Project Brief

A plain-English + technical walkthrough of what we are building, where we are, what it costs, and whether it scales.

---

1. What Makoya is (the 30-second version)

Makoya sells and runs an embeddable web-accessibility widget.

A website owner pastes **one line of code** onto their site. Their visitors then get a floating button that opens a panel of **18 accessibility controls** — bigger text, higher contrast, stop-motion, reading ruler, readable font, read-aloud, big cursor, and more. It works in **4 languages** and ships with **9 one-click profiles** (e.g. “dyslexia”, “low vision”, “ADHD”).

But the widget is only one third of the product. We are really three things sharing one backend:

Surface	What the user sees	What it really is
Scanner	”Scan my site for accessibility issues” → a score + plain-English fixes	A real WCAG 2.0/2.1/2.2 engine. It is both our **product core** and the **top of our sales funnel**.
Widget	A `<script>` they paste once	A tiny loader + a versioned core bundle. Applies 18 preferences to any page without breaking it.
Dashboard + Admin CRM	A place to customize their widget, see their scan report, manage billing	A Next.js app on Supabase with real login + per-tenant data isolation.

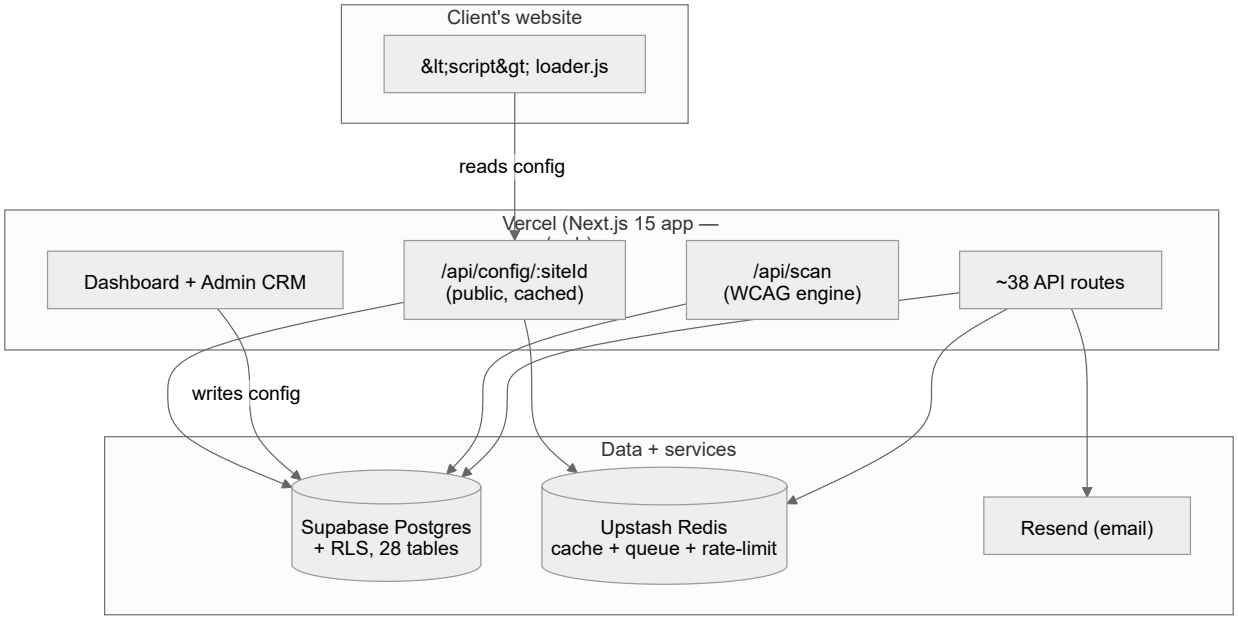
OUR GOAL

Become the **trustworthy** accessibility layer for the web — the opposite of the “overlay” products that got sued for making false compliance claims and breaking screen readers. We win on **honesty** (we offer *tools*, never “guaranteed compliance”), a **real scanner**,

and a widget that is **engineered not to break the host site**.

---

2. How the pieces fit (architecture)



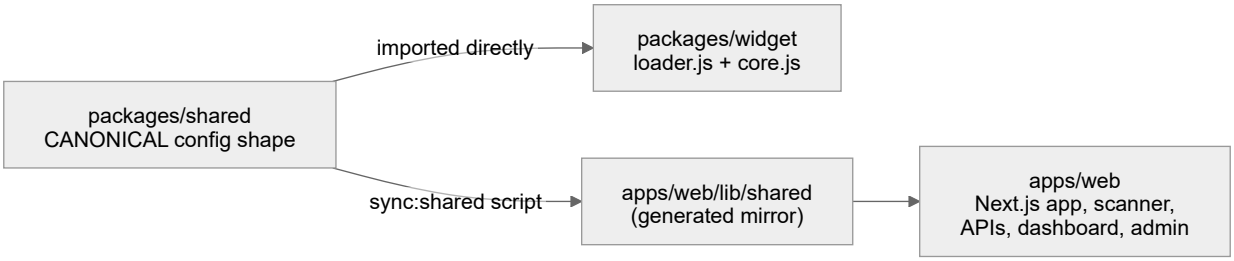
Makoya system architecture

The one rule that ties it together: config flows **one way**. The dashboard *writes* a site's config to the database; the widget loader only ever *reads* a public, safe-to-expose JSON. Both ends share one TypeScript type package (`@makoya/shared`) so they can never disagree about the shape.

---

3. How we work — the monorepo + workspaces

We use **npm workspaces** (one repo, many packages). This is what keeps the widget and the app in sync.



Workspace layout

- `packages/shared` — the single source of truth for the widget config.
- `packages/widget` — the embeddable widget. Two bundles: a tiny stable `loader.js` the client pastes once, and a versioned `core.js` we can upgrade forever without the client changing their snippet.
- `apps/web` — the Next.js 15 / React 19 app: dashboard, admin CRM, the scanner engine, all the APIs, auth.
- CI **fails the build** if the generated mirror drifts from canonical, so the two halves can't silently diverge.

Multi-agent workflow note: the founder often runs 2–3 AI coding sessions at once. We enforce *one worktree = one branch = one agent*, tracked on a coordination board in `docs/STATUS.md`, so parallel work never collides.

---

4. The tools we use, and *why*

Everything is on a **free tier today**; the paid upgrades are deliberate, small, and listed in section 7.

Tool	Role	Why this one	Status
Vercel	Hosting + serverless functions	Zero-config Next.js, auto-scaling functions, global edge	✅ Live
Supabase	Postgres database + Auth	Real multi-tenant DB with **Row-Level Security** (cross-tenant reads impossible at the DB layer) + managed auth	✅ Live
Upstash Redis	Cache + job queue + rate-limit	Serverless Redis. Powers our config cache, scanner queue, and abuse protection. **No-ops safely if unconfigured**	✅ Live
Resend	Transactional email	Simple, reliable; a send failure never blocks a lead capture	✅ Live
Sentry	Error monitoring	Catches server/edge/client crashes; routed through one seam so it's swappable	✅ Wired
PostHog	Product analytics	See what users actually do; privacy-scrubbed (we deny-list sensitive params)	✅ Wired
Playwright + axe-core	Scanner engine	Industry-standard accessibility testing, run in a real headless browser	✅ Live
Stripe	Billing (chosen processor)	Schema + plan catalog already built; just needs the account + keys wired	⛔ Not yet

Design principle: every external service is behind a **single seam** (`lib/observability.ts`, `lib/rate-limit.ts`, `lib/config-cache.ts`) and **fails open**. If Redis or Sentry is down, the product keeps working — it just loses the optimization, never the function.

---

5. The database + backend, simplified

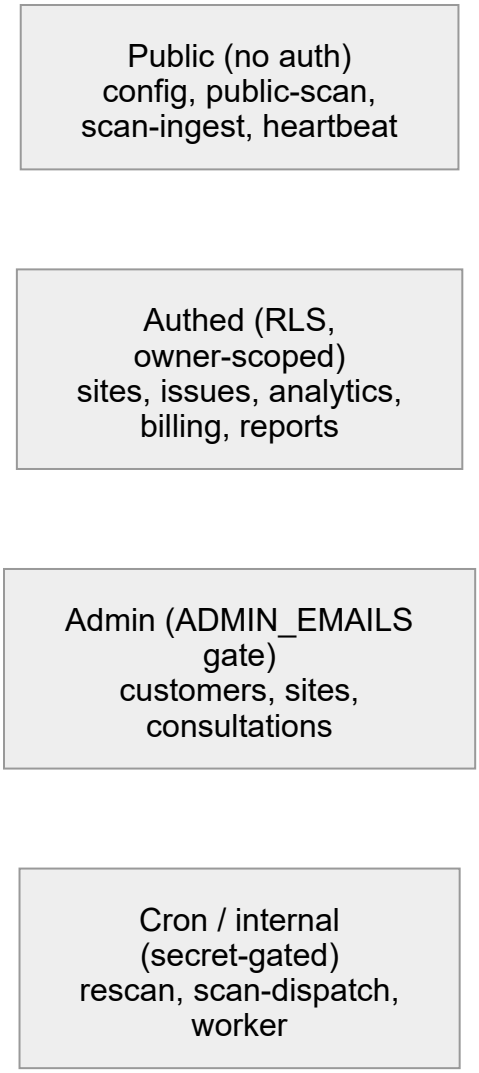
DATABASE — SUPABASE POSTGRES, 28 TABLES, ALL RLS-ENABLED

Database table groups

The load-bearing idea is **Row-Level Security (RLS)**: every row is owned by an auth user, and the database itself refuses to return another tenant's rows. We don't rely on app code to remember to filter — the DB enforces it. We even verify this in CI: a script spins up a throwaway Postgres, applies every migration, and asserts cross-tenant reads are denied (10/10 checks).

Two tables (`leads`, `consultation_requests`) are service-role-only — RLS on, *no* policy. Only our trusted server can touch them; owners never see each other's leads.

BACKEND — ~38 API ROUTES IN THREE RINGS



API surface

The **service key never reaches the browser**. The widget only ever fetches a public, read-only config JSON that exposes safe display fields.

---

6. Can we handle 10,000 users at once? (concurrency + scalability)

This is the most important question, so here is the honest answer in three parts.

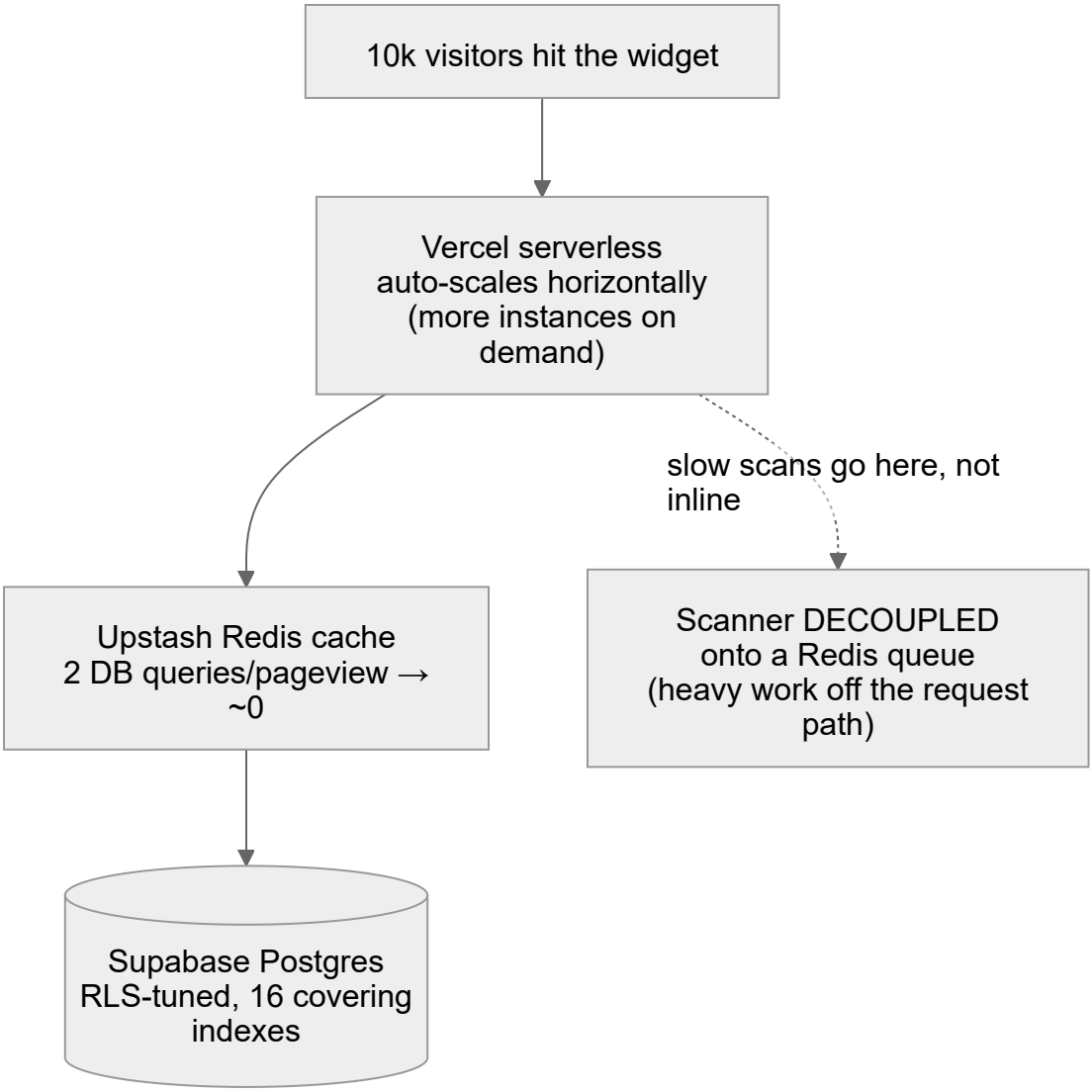
WHAT WE HAVE ACTUALLY PROVEN

We ran a **live load test against production** `/api/config` — roughly **13,700 requests**:

Metric	Result
Cold request (cache miss → Postgres)	850–1160 ms
Warm request (cache hit → Redis)	**p50 ~270 ms / p95 ~300 ms**
Speed-up from the cache	**3–4×**
Errors through 200 concurrent	**0% — no 5xx**

At 200 concurrent the warm latency is **network-bound**, not compute-bound — the server isn’t breaking a sweat. That is the signal that the architecture is sound.

WHY THE ARCHITECTURE SCALES (THE DESIGN CHOICES THAT MATTER)



The scalability stack

1. **Stateless + auto-scaling.** Vercel functions add instances automatically under load. No single server to fall over.
2. **Read-through cache on the hot path.** The widget's config request — the one that fires on *every page view of every client site* — hits Redis, not Postgres. Two DB queries per view became roughly zero.

3. **The expensive thing is off the request path.** A WCAG scan opens a real Chromium browser — too heavy to do inside a web request at scale. So scheduled scans go onto a **Redis queue** with a visibility-timeout claim, dedupe, poison/dead-letter handling, and a worker that runs one browser per instance. Interactive scans stay synchronous but are **rate-limited and concurrency-capped**.
4. **The database was tuned for growth.** We rewrote RLS policies for performance and added 16 covering indexes specifically so we can grow to hundreds of thousands of client sites.
5. **Everything fails open.** Cache/queue/monitoring outages degrade gracefully; they never take the product down.

THE HONEST GAPS BEFORE WE *PROMISE* 10K SIMULTANEOUS

Gap	Why it matters	Fix
Tested to **200 concurrent**, not 10k	We have strong evidence, not a 10k proof	Run a staged 1k → 5k → 10k load test
On **Vercel + Supabase free tiers**	Free Supabase **auto-pauses**; free Vercel caps cron + concurrency	Upgrade to **Supabase Pro ($25/mo)** + **Vercel Pro**
Postgres connection limits under burst	Many serverless instances × DB connections can exhaust the pool	Use Supabase's connection pooler (PgBouncer) — available on Pro
Scanner throughput	Per-minute dispatch needs Vercel Pro (free tier = daily cron only)	Enable Pro, restore per-minute cron

Bottom line for the meeting: *The architecture is built to handle 10k — stateless auto-scaling, a cache on the hot path, a decoupled job queue, and a tuned database. We've proven 0% errors and ~~270ms warm responses at our tested load. To confidently certify 10k simultaneous, we flip on the paid tiers~~ ($25–50/mo), turn on connection pooling, and run a staged load test. No re-architecture required.*

---

7. What's paid / still to integrate

Item	Cost	Unblocks	Priority
Stripe account	Free acct, ~2.9% + 30¢/txn	Real payments + plan gating (schema already built)	🔴 High — this is the gap between demo and revenue
Supabase Pro	$25/mo	Stops free-tier auto-pause; connection pooling; more headroom	🔴 High
Vercel Pro	~$20/mo	Per-minute scanner cron; higher concurrency	🟠 Medium
Enable Supabase leaked-password toggle	Free, 30s	Security hardening	🟢 Quick win
Founder's own booking embed	—	Replaces the consultation placeholder	🟢 When ready
Optional later	varies	Stripe Tax, pen-test, BrowserStack	🟣 Future

Note: Stripe is a *processor*, not a Merchant-of-Record, so **VAT/tax is on us** (Stripe Tax add-on or manual).

---

8. Where we are — phases

Roadmap status

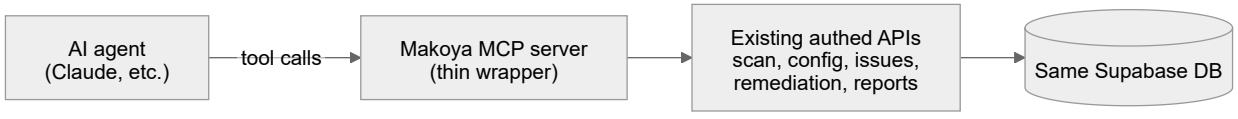
Phase	State	Detail
0 — Foundation	✅ Done	Monorepo, shared config contract, widget loader/core split
1 — Revenue loop	✅ Live	Public scan → score + plain-English issues → email capture → lead → admin CRM. PDF export live.
H — Bulletproofing	✅ Done	SSRF defense, Zod validation everywhere, hardened RLS, 600+ tests, security CI gates, **scalability foundation** (cache, queue, index tuning) — all live
2 — Money	⛔ Blocked	Stripe checkout + signed/idempotent webhook → plan gating. **Only needs the Stripe account.**
3 — Demo polish	🔶 In flight	Landing ✅; remaining = founder's booking embed + analytics dashboards
4 — Features	🟣 Next	Scheduled monitoring + "score dropped" alerts, AI remediation (human-confirmed), WordPress plugin
5 — Enterprise	🟣 Future	White-label, human audits, VPAT/ACR, SSO

Live today (verified in production): the widget, the real scanner, the full public funnel, PDF export, Supabase auth + RLS, email, rate-limiting, error monitoring, and **widget licensing enforcement** (only paying, domain-allowlisted sites load the widget).

---

9. The future: making the widget an MCP (yes, we can)

The founder's direction is to evolve the widget into an **MCP server** (Model Context Protocol — the standard that lets AI agents call tools). **The architecture already supports this**, because everything is built as clean, authenticated, JSON API surfaces.



Widget-as-MCP — the natural next layer

What an agent could do via MCP: *”scan this site”*, *”what are the top accessibility issues?”*, *”turn on the dyslexia profile for this client”*, *”generate the monthly report”*. Because our routes are already typed, auth-gated, and tenant-isolated, the MCP server is mostly a **thin translation layer** on top of what exists — not a rewrite.

Same story for **adding more widget features**: the loader/core split means we ship new accessibility tools by publishing a **new core bundle**. The client’s pasted snippet never changes. We went from 15 → 18 features this way already.

---

10. One-paragraph summary for the room

> Makoya is a three-in-one accessibility product — a real WCAG scanner, an embeddable widget with 18 preferences, and a multi-tenant dashboard + CRM — built on Next.js, Supabase (Postgres + RLS), Upstash Redis, and Vercel. It is **live in production**, security-hardened, and load-tested at **0% errors with ~270ms warm responses**. The architecture is **stateless, cached, queue-decoupled, and index-tuned** specifically to scale to tens of thousands of users without a rewrite — the remaining work to certify 10k is flipping on paid tiers (~$45/mo) and a staged load test. The single thing standing between us and real revenue is wiring **Stripe** (the schema is already built). Our future — turning the widget into an **MCP** for AI agents and adding more features — is a natural extension of the clean API surfaces we already have.